



National University
of computer and emerging sciences

Brick Breakers

Iteration Documentation

Iteration 3

#	Name	Roll Number
1	Ermish Tabassum	24i-3180
2	Sawaira Fareed	24i-3043
3	Hanzala Ahsan	24i-3150
4	Ahmed Tanveer	23i-3019

Contents

1. Project Overview	4
2. Screen Descriptions	4
Screen 1: Home Screen	5
Screen 2: Name Input Screen.....	6
Screen 3: Main Menu Screen.....	7
Screen 4: Instructions Screen.....	9
.....	9
Screen 5: High Score Screen.....	11
Screen 6: Game Screen Layout — Level 1.....	12
Screen 7: Game Screen Layout — Level 2.....	14
Screen 8: Game Screen Layout — Level 3.....	15
Screen 9: Level Complete Screen	17
Screen 10: Game Over Screen	19
Screen 11: Win Screen.....	20
3. Screen Flow Diagram	22
Iteration 1 Screen Flow	22
.....	22
Iteration 2 Screen Flow	23
Iteration 3 Screen Flow	24
Navigation Summary — Iteration 3:.....	25
4. Technical Architecture.....	26
4.1 Game Loop Architecture.....	26
4.2 Collision Detection	27
4.3 Multi-Level System	28
4.4 Bonus / Power-Up System.....	29
4.5 Lives System.....	30
4.6 Sound Effects System	31
5. Iteration 3 Scope & Marking	32
5.1 Multi-Level System (10 Marks).....	32
5.2 Power-Up System (10 Marks)	33
5.3 Complete Game Flow (5 Marks)	33
5.4 Game Polish & Stability (5 Marks)	34
5.5 Bonus Marks (+10)	34

27. Data Structures.....	35
27.1 Paddle & Ball Variables.....	35
27.2 Brick State Arrays.....	36
27.3 Game State Variables.....	36
27.4 Bonus / Power-Up Variables	36
27.5 String & Display Buffers	37
27.6 Font Table	37
27.7 Sound Variables	38
28. Procedure Descriptions	39
28.1 Screen Procedures	39
28.2 Game Screen Procedures	40
28.3 Game Logic Procedures	40
28.4 Bonus / Power-Up Procedures	41
28.5 Drawing Utility Procedures	41
28.6 HUD Update Procedures.....	42
28.7 Sound Procedures.....	42
28.8 Delay Procedures	43
29. Known Issues	44
30. Task Division	45

1. Project Overview

Brick Breakers is a game in assembly language that is played in DOS VGA mode (320x200, Mode 13h). It's a classic arcade shooter where the player takes control of a paddle and bounces a ball, smashing bricks that form grids on various levels.

Iteration 3 takes the working prototype of Iteration 2, which was a single level prototype, and makes it into a complete, working system for a game. The main additions to Iteration 3 include three different levels of difficulty and different brick patterns, a fully functional bonus/power-up system, a complete game cycle, including start, play, level progression, win, game over, and replay, as well as PC speaker sound effects.

Language	Mode	Resolution
x86 Assembly (MASM)	VGA Mode 13h	320 x 200 pixels

2. Screen Descriptions

Complete game has 11 screens. Each screen is detailed below in terms of its purpose, visual elements, technical aspects and navigability. The following 5 screens (Home, Name Input, Main Menu, Instructions, High Scores) remain the same as in Iteration 2. Screens 6-11 are either new or modified screens that were added in Iteration 3.

Screen 1: Home Screen



2.1 Home Screen

The game starts with the Home Screen. Sets the visual mood and theme with a complete brick grid backdrop (like in real-life play), an title box in the center with an ornamental double framed design, the game title in bright magenta type, team members names at the bottom, and the message: Press any key.

Visual Elements

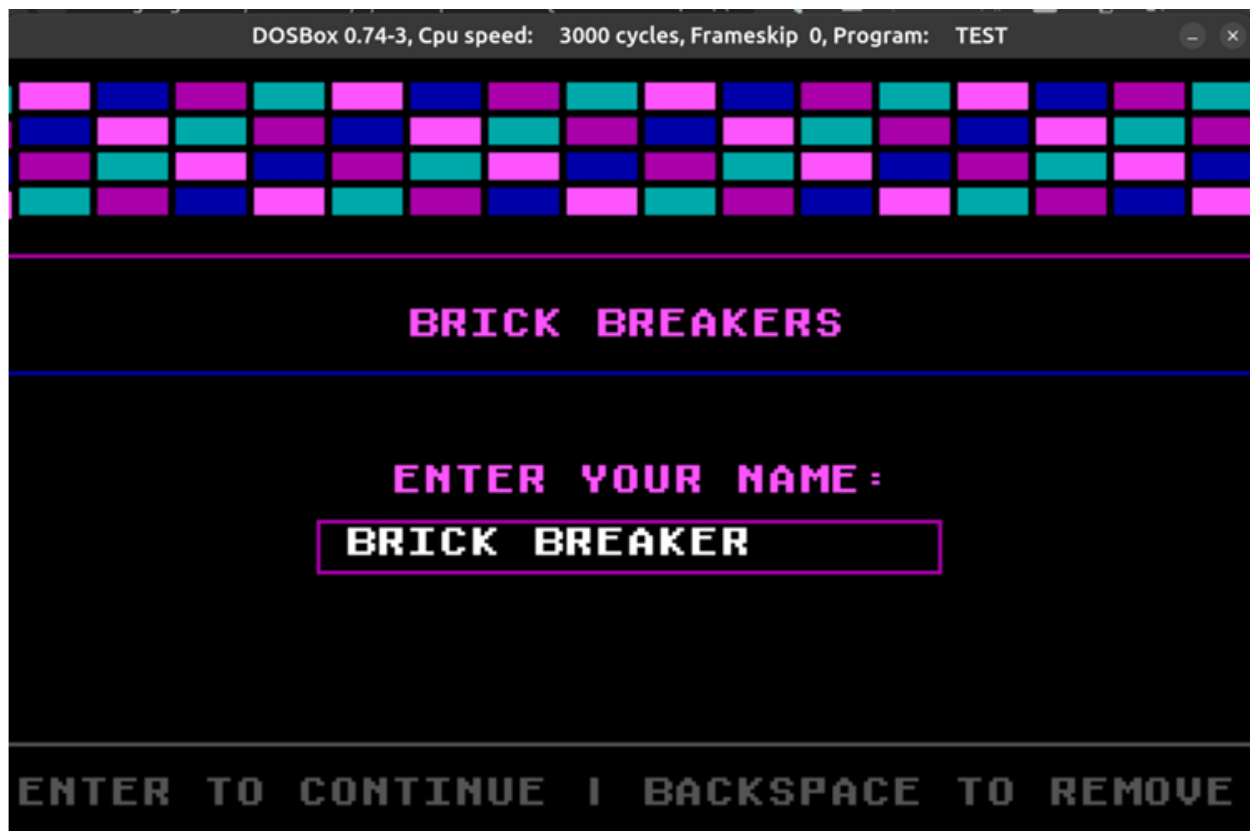
- Game name: BRICK BREAKERS in large magenta custom font, horizontally centred.
- Color-cycling bricks (magenta, blue, purple, cyan) in a 4 row x 16 column animated brick background.
- Border frame in the shape of two rectangles drawn around the title in cyan.
- Prompt text: 'press any key to continue' in purple, positioned under the title.
- Team credits line at the bottom: 'Ermish Sawaira Ahmed Hanzala | SE-C' in dark gray.

- Horizontal separator line in dark gray at the bottom.

Navigation Flow

Press any key: Proceeds to the Name Input Screen.

Screen 2: Name Input Screen



2.2 Name Input Screen

The Name Input Screen allows the player to type in their name to begin the game. It makes sure to retain the BRICK BREAKERS name at the top for uniformity, and has a distinctively defined input field. The player types and gets immediate feedback, showing each character in the input box.

Visual Elements

- Game title retained at top in magenta for visual continuity.

- Horizontal separator lines above and below the input area.
- Label text: 'Enter Your Name:' in magenta above the input box.
- Input box: a rectangle with a purple border and black interior.
- Live character display: typed characters shown in white inside the input box.
- Footer instruction: 'enter to continue | backspace to remove' in gray at the bottom.

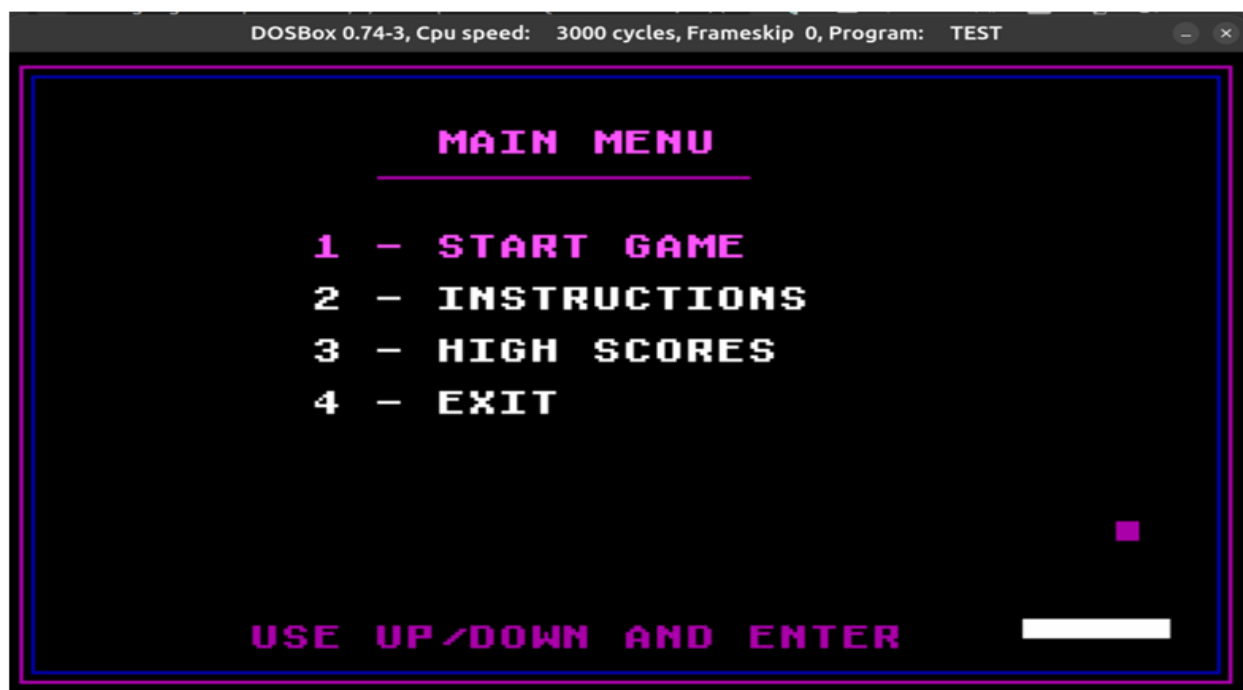
Functional Behavior

- Typed characters are concatenated into name_buf (maximum 20 characters).
- Backspace (ASCII 8) removes the last character from the buffer and clears the display.
- Enter key (ASCII 13) validates the name, copies it to name_stored and gs_name, then proceeds.
- If the buffer is empty when Enter is pressed, the player is kept on the input screen.

Navigation Flow

Enter key: Name stored; proceeds to the Main Menu Screen.

Screen 3: Main Menu Screen



2.3 Main Menu Screen

The Main Menu Screen is the main navigation screen of the game. Has a neon double border frame, four menu items and is fully keyboard navigable with highlighting. Ambient decoration – animated ball bounces up and down in the right-side panel of the menu.

Visual Elements

- Neon double-border frame: outer purple (5px), inner blue (7px) to create a glow effect.
- Centered title: MAIN MENU in magenta with a horizontal underline.
- Four menu items: 1 - START GAME, 2 - INSTRUCTIONS, 3 - HIGH SCORES, 4 - EXIT.
- Selection highlight: the currently selected option is displayed in magenta with a '>' arrow.
- Navigation hint at the bottom: 'Use UP/DOWN and ENTER' in purple.
- Animated bouncing ball (magenta rectangle) in the lower-right corner of the frame.

Functional Behavior

- Up/Down arrow keys move the selection cursor through the four options.
- Enter key navigates to the chosen screen: Game Layout, Instructions, or High Scores.
- Selecting Exit (option 4) or pressing ESC returns to the OS.

Navigation Flow

Up/Down: Move selection | Enter: Navigate to selected screen | ESC: Exit program.

Screen 4: Instructions Screen



2.4 Instructions Screen

All game information is displayed on the Instructions Screen in two pages so as not to clutter the screen. The neon frame and title HOW TO PLAY are shared between both pages. Page indicators (1/2) and (2/2) are displayed in the upper-right corner.

Page 1: Controls, Objective & Lives

- Controls section: a two-column table listing keys (Left Arrow / A, Right Arrow / D, Space) and their corresponding actions (Move Paddle Left, Move Paddle Right, Bounce the Ball).
- Objective section: states that the player must break all bricks to advance to the next level.
- Lives section: indicates the player starts with 3 lives and loses one each time the ball exits the bottom of the screen.
- Bottom bar: 'ESC-Back' on the left, 'PRESS ENTER FOR MORE >>' on the right.

Page 2: Bonuses

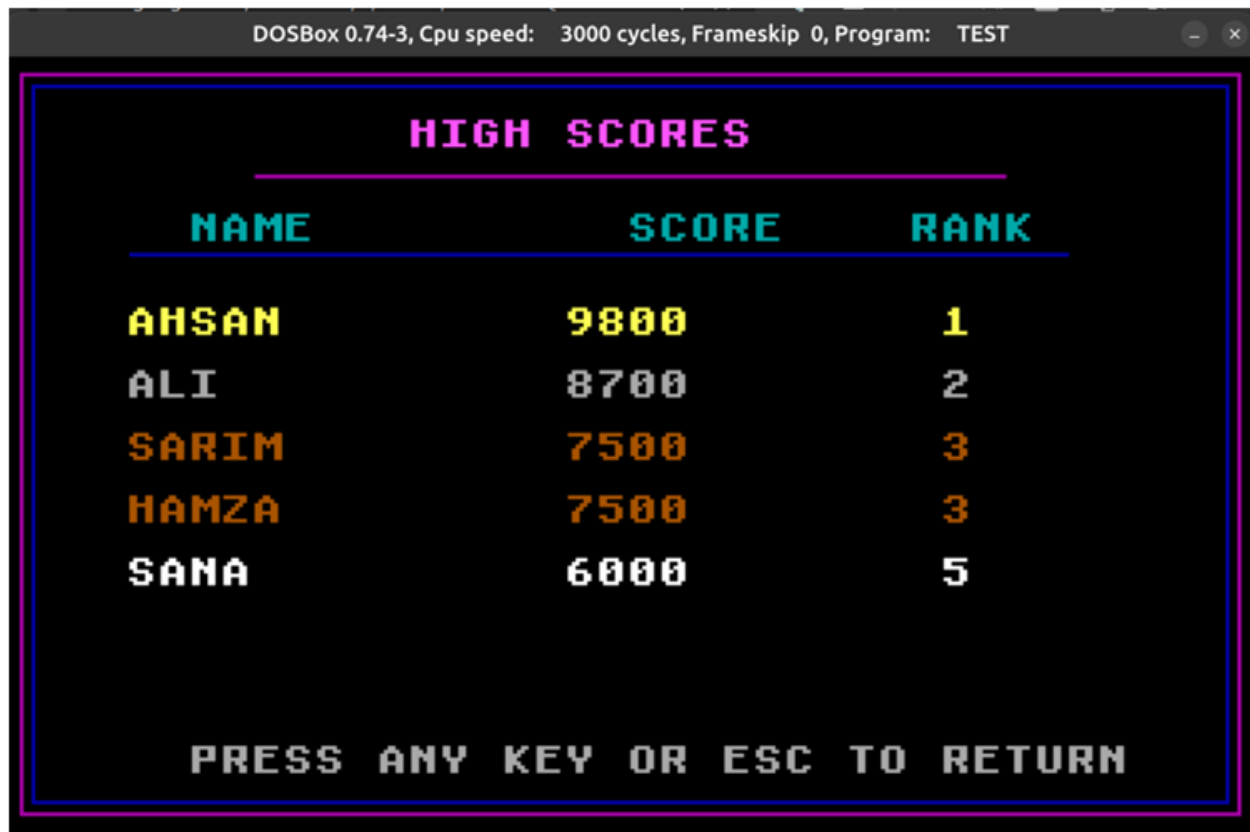
- Bonuses section: explains that falling items are dropped when bricks are destroyed.
- Color-coded two-column table listing all five bonus types (Cyan, Red, Green, Yellow, Purple) and their effects (Slow Ball, Fast Ball, +1 Life, Wide Paddle, Narrow Paddle).
- Note: 'Only one bonus active at a time' displayed below the table.
- Bottom bar: 'ESC-Back' on the left, 'ENTER for page 1 ->' on the right.

Navigation Flow

Page 1: Enter → Page 2 | ESC → Back to Main Menu.

Page 2: Enter → Page 1 | ESC → Back to Main Menu.

Screen 5: High Score Screen



2.5 High Score Screen

The High Score Screen is a list of 5 static leader board scores. The information is statically coded in the .Implementation of no file I/O operation in this iteration (DATA segment). Entries are color-coded by number for ease of visual identification.

Visual Elements

- Title: HIGH SCORES in magenta, centered, with a horizontal underline.
- Table header row: NAME, SCORE, and RANK columns in cyan.
- Five entries with color-coded rankings: gold (1st), gray (2nd), orange (3rd/4th), white (5th).
- Return message at the bottom: 'Press any key or ESC to return'.

Navigation Flow

Any key or ESC: Returns to the Main Menu Screen.

Screen 6: Game Screen Layout — Level 1



2.6 Game Screen Layout (Level 1)

The entry level of the game, Iteration 2 carry-over, is Level 1. It shows a regular grid of 5 rows and 15 columns of bricks. Ball velocity is moderate and the paddle's width is set to the default of 70 pixels. This player needs to clear all 75 bricks in order to go to Level 2.

HUD (Heads-Up Display)

- Top bar: a purple-filled rectangle spanning the full width (320px) at 28px height.
- Score (top-left): 'SCORE: 0000' in white, updated live as bricks are broken.
- Lives (top-center): 'LIVES:' label followed by up to 3 red heart icons (custom bitmap).
- Level (top-right): 'LEVEL: 01' in white.
- Player name (second row): 'PLAYER:' label followed by the name entered on the Name Input Screen.

Brick Layout

- 5 rows x 15 columns of bricks, color-cycling through magenta, blue, purple, and cyan.
- All 75 bricks start alive (brick_state array initialized to all 1s).
- Bricks start at Y=48; each row is 11px tall (8px brick + 3px gap); each column is 20px wide.

Game Objects

- Ball: a 6x6 white rectangle; starts at paddle_x + 32, Y=170; ball_dx=1, ball_dy=-1; ball_speed=6.
- Paddle: a 70x8 white rectangle at initial position (125, 185).

Bonus System

- A colored 8x8 square falls toward the paddle every third brick break.
- Bonus type cycles through: Slow Ball (cyan), Fast Ball (red), Extra Life (green), Wide Paddle (yellow), Narrow Paddle (purple).

Navigation Flow

ESC: Returns to the Main Menu Screen. Break all bricks: transitions to Level 2.

Screen 7: Game Screen Layout — Level 2



2.7 Game Screen Layout (Level 2)

The Tunnel Layout is a 6 by 15 brick grid, with a fixed vertical gap in the middle three columns (6,7,8) of the grid, which is Level 2. This results in a crease the ball can pass through, altering the ball's trajectory and making collisions difficult to anticipate. The ball speed is lowered to `ball_speed=4` to somewhat offset the added complexity of the layout.

HUD

- Identical to Level 1 HUD; level indicator updated to 'LEVEL: 02'.
- Score carries over from Level 1; lives carry over from Level 1.

Brick Layout — Tunnel

- 6 rows x 15 columns; columns 6, 7, and 8 are permanently empty (`brick_state2` value = 2 for gap cells).
- 10 bricks per row (columns 0–5 and 9–14), totalling 60 breakable bricks.
- Row colors alternate: even rows cyan/teal (03h), odd rows magenta (0Dh).

- Gap columns are painted black to ensure a clean visual tunnel.

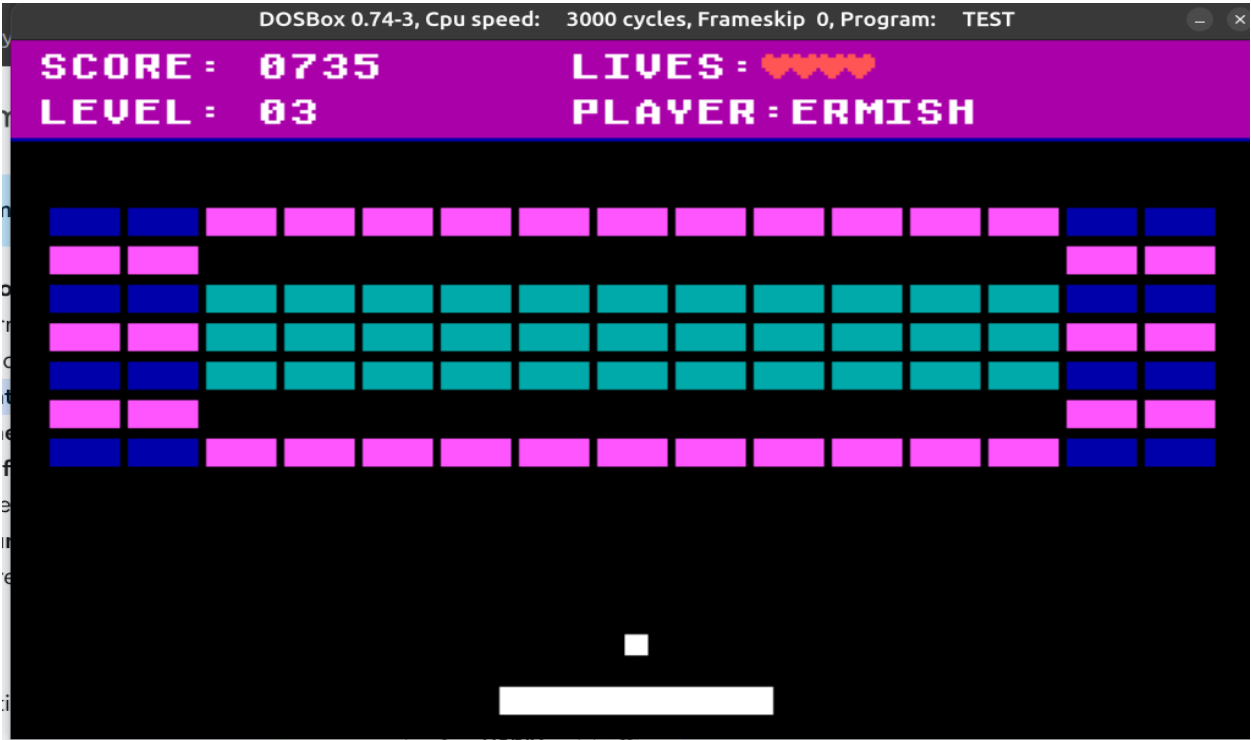
Difficulty Increase

Parameter	Level 1	Level 2
Brick Rows	5	6
Breakable Bricks	75	60
Ball Speed (delay)	6	4 (faster)
Ball dx/dy	1 / -1	1 / -1
Layout Complexity	Uniform grid	Tunnel gap in center

Navigation Flow

ESC: Returns to the Main Menu Screen. Break all 60 breakable bricks: transitions to Level 3.

Screen 8: Game Screen Layout — Level 3



2.8 Game Screen Layout (Level 3)

The Fortress Layout, a 7-row by 15-column asymmetric brick pattern made with Layers 3 is used to make an outer wall and an inner block with intentional voids between. The speed of the ball is considerably accelerated (ball_speed=2, which means that it requires less delay cycles), and ball_dx/ball_dy are both equal to 2, which means that the ball moves diagonally with a higher speed. This is the last and hardest level.

HUD

- Identical HUD structure; level indicator updated to 'LEVEL: 03'.
- Score and lives carry over from Level 2.

Brick Layout — Fortress

- 7 rows x 15 columns. Alive bricks form two zones:
 - Outer Wall: columns 0–1 (left) and columns 13–14 (right) for all 7 rows; plus row 0 (top) and row 6 (bottom) spanning all columns.
 - Inner Block: rows 2–4, columns 2–12 (a solid 3-row by 11-column filled rectangle).
 - Empty zones: rows 1 and 5 (between outer wall and inner block) serve as passage corridors.
- Colors: outer left/right wall alternates dark blue (01h) and magenta (0Dh) per row; top/bottom rows are magenta (0Dh); inner block is teal/cyan (03h).

Difficulty Increase

Parameter	Level 2	Level 3
Brick Rows	6	7
Breakable Bricks	60	~75 (fortress shape)
Ball Speed (delay)	4	2 (much faster)
Ball dx/dy	1 / -1	2 / -2 (diagonal speed x2)

Layout Complexity	Tunnel gap	Fortress with corridors
-------------------	------------	-------------------------

Navigation Flow

ESC: Returns to the Main Menu Screen. Break all alive bricks: triggers the Win Screen (is_final_win = 1).

Screen 9: Level Complete Screen



2.9 Level Complete Screen

The Level Complete screen will appear on its own if all the breakable bricks have been removed from a level, but the level is not the last one. It offers player a summary of the session and allows him to either advance to the next level or return to the Main Menu.

Visual Elements

- Double-bordered frame: outer purple, inner blue (same neon style as Main Menu).
- Title: 'LEVEL COMPLETE!' in magenta, centered near the top.
- Inner stats box (bordered rectangle) containing four labeled rows: Final Score, Level Reached, Bricks Left, and Lives Remaining — all pulled from live game state.
- Two options below the stats box: '[ENTER] Next Level' in purple and '[ESC] Main Menu' in gray.

Functional Behavior

- `is_final_win` is set to 0 before this screen is shown for non-final levels.
- Score, lives, and level number values are read directly from live game variables.
- On ENTER: the screen returns to the caller (`CheckAllBricksDead`), which then calls `TransitionToLevel2` or `TransitionToLevel3` respectively.

Navigation Flow

ENTER: Proceeds to the next level. ESC: Returns to the Main Menu Screen.

Screen 10: Game Over Screen



2.10 Game Over Screen

The Game Over Screen will appear on its own when the player's lives are depleted. It displays an overview of the finished session in a double-bordered frame and provides two navigation options.

Visual Elements

- Title: 'GAME OVER' drawn character by character in large red text (color 0Ch), horizontally centered.
- Double-bordered outer frame: outer purple, inner blue.
- Inner stats box: a bordered rectangle with four labeled rows — Final Score, Current Level, Bricks Left, and Lives Remaining — all populated from live game state.
- Two options below the stats box: '[ENTER] Restart Game' in purple and '[ESC] Main Menu' in purple.

Functional Behavior

- The screen is called from within MoveBall when lives_count reaches 0.
- Bricks Left is computed by counting entries equal to 1 in brick_state at the time of death.
- On ENTER: resets score to 0, lives to 3, current_level to 1, all bonus variables, paddle_width to 70, ball_speed to 6, all 75 brick_state entries to 1, then calls GameScreenLayout fresh.
- On ESC: returns via standard RET, unwinding back to MainMenuScreen.

Navigation Flow

ENTER: Resets all state variables and restarts a fresh game from Level 1. ESC: Returns to the Main Menu Screen.

Screen 11: Win Screen



2.11 Win Screen

The Win Screen will appear if the player completes all the bricks from the last level (Level 3). It's a conditional branch in LevelCompleteScreen with `is_final_win` set to 1, changing the title to 'YOU WIN!' and changing the options to restart or return to menu.

Visual Elements

- Double-bordered frame: outer purple, inner blue (same frame as Level Complete and Game Over).
- Title: 'YOU WIN!' drawn character by character in magenta (0Dh), horizontally centered.
- Inner stats box: identical layout to Level Complete — Final Score, Level Reached (shown as 3), Bricks Left, and Lives Remaining.
- Two options below the stats box: '[ENTER] Play Again' in purple and '[ESC] Main Menu' in gray.

Functional Behavior

- `is_final_win` is set to 1 in CheckAllBricksDead before calling LevelCompleteScreen.
- On ENTER: fully resets all game state (score, lives, level, bonuses, bricks) and calls GameScreenLayout, starting a fresh game from Level 1.
- On ESC: returns to the MainMenuScreen.

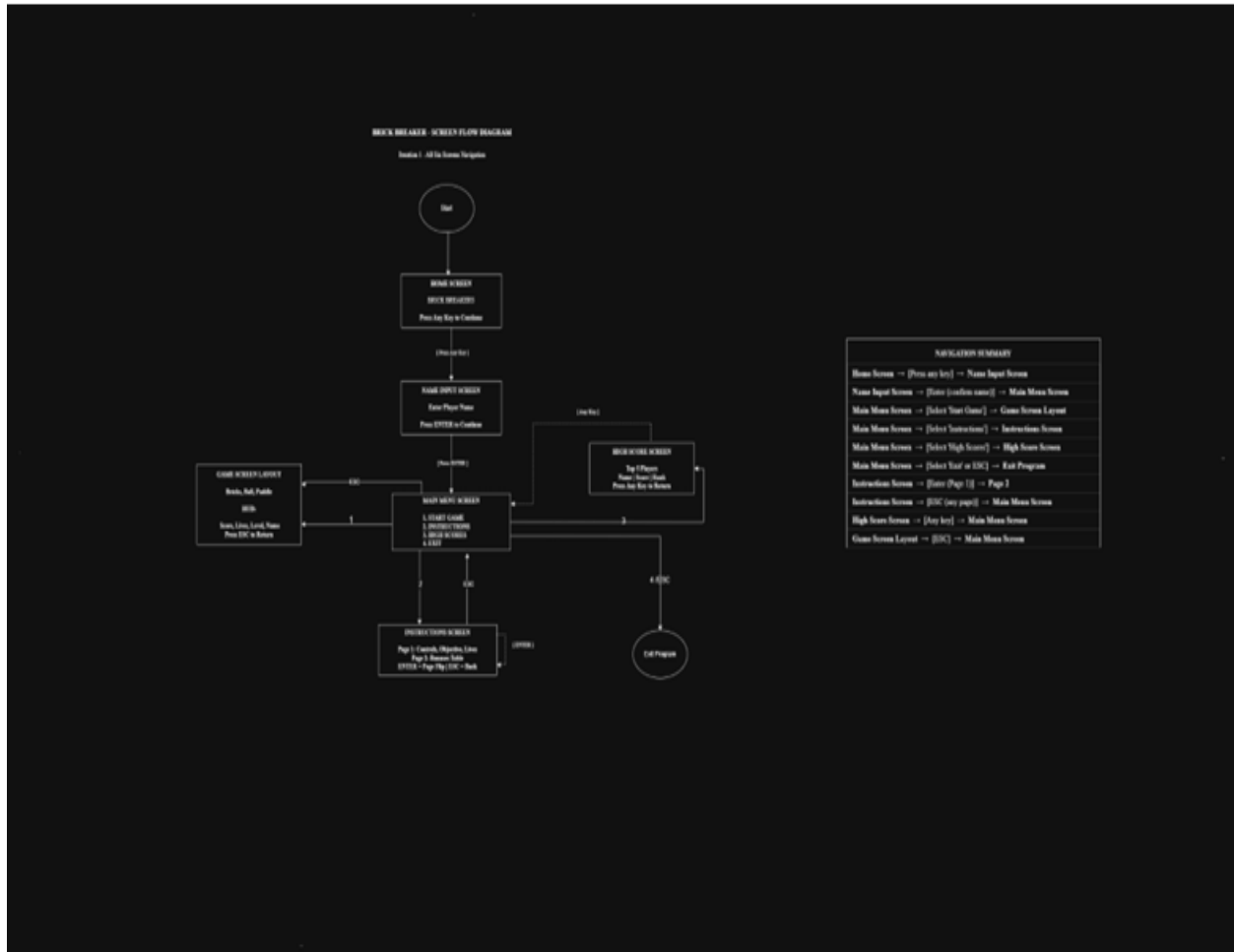
Navigation Flow

ENTER: Resets all game state and restarts from Level 1. ESC: Returns to the Main Menu Screen.

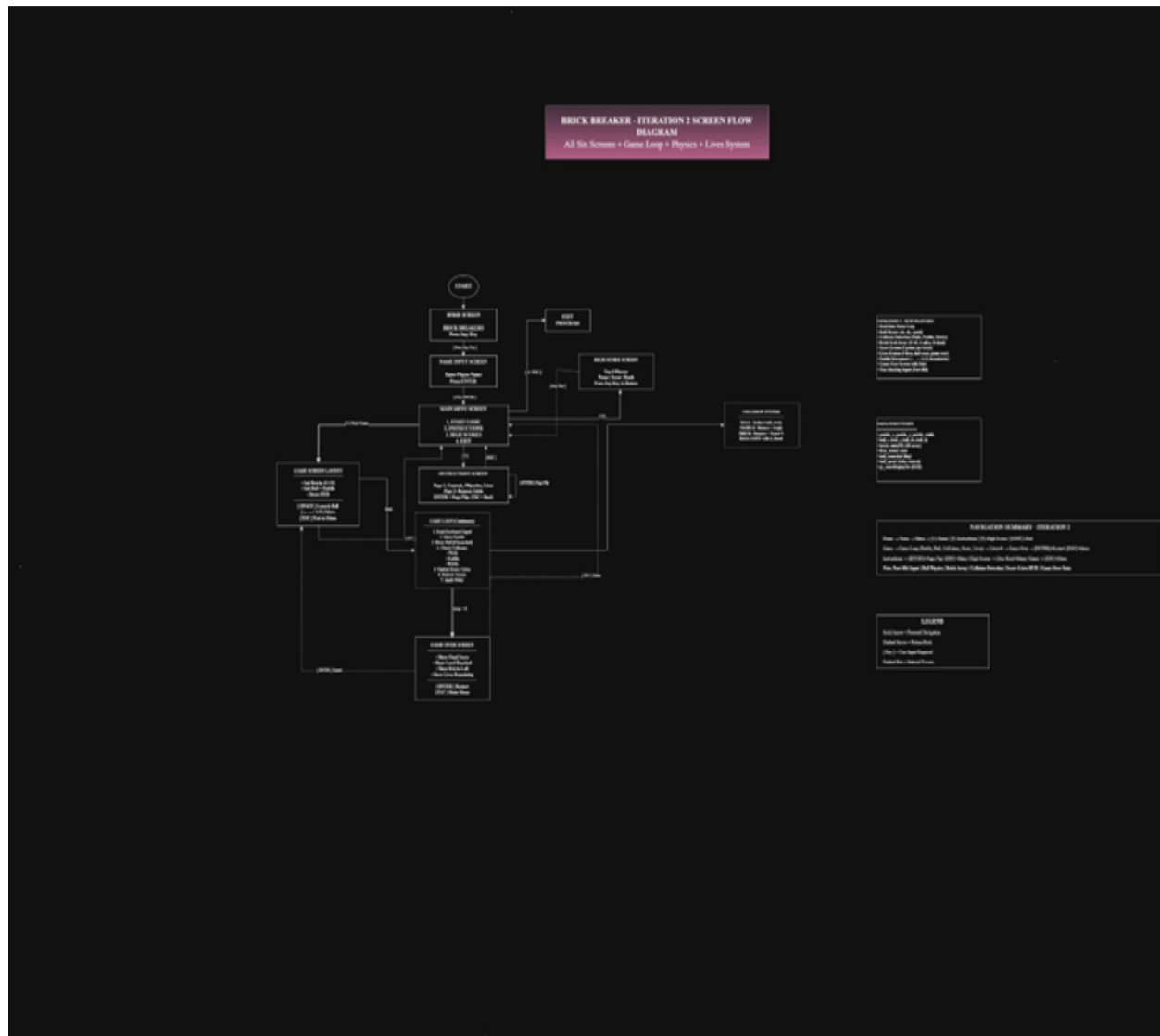
3. Screen Flow Diagram

The following diagrams describe the complete navigation flow between all screens for Iteration 3.

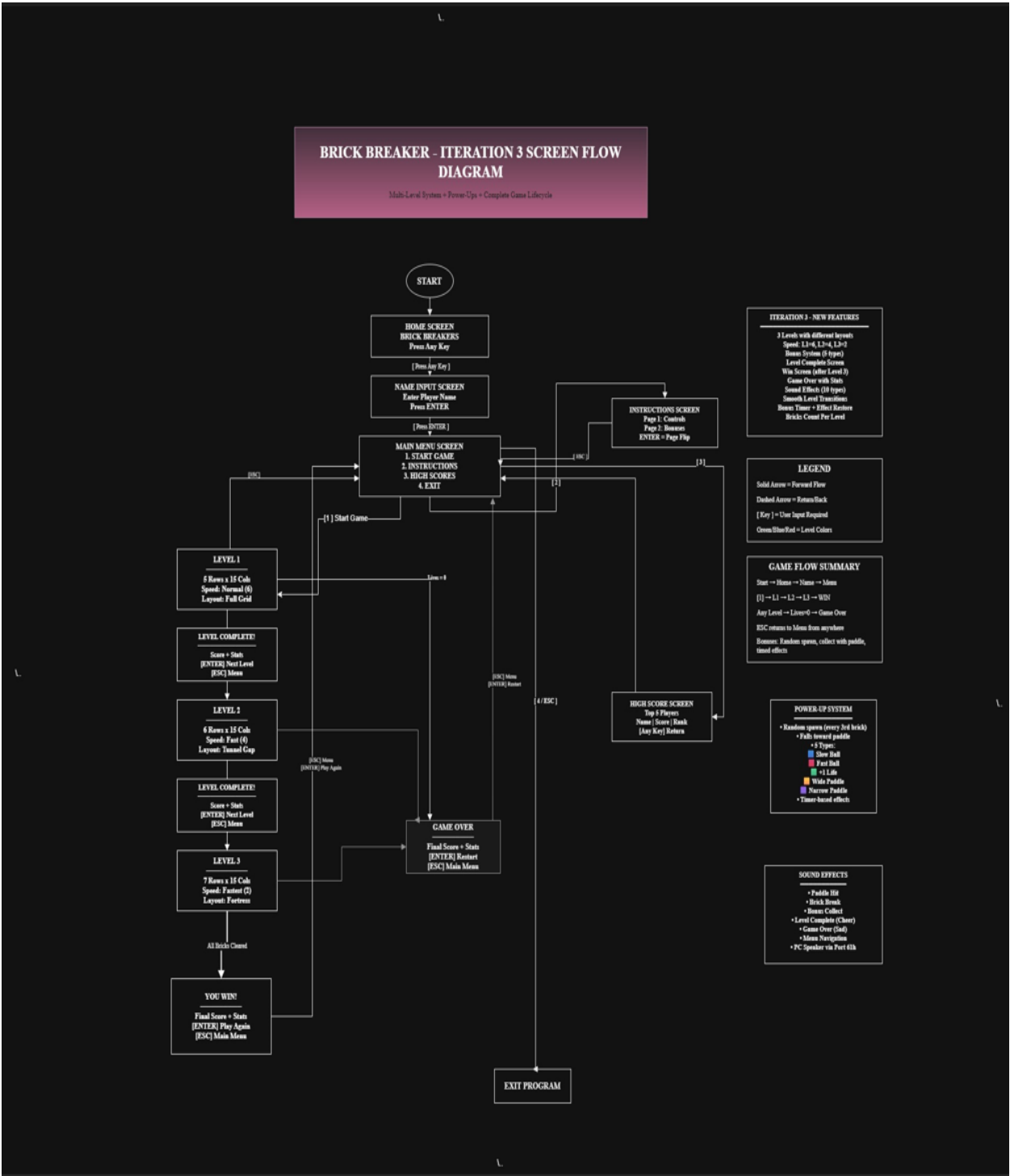
Iteration 1 Screen Flow



Iteration 2 Screen Flow



Iteration 3 Screen Flow



Navigation Summary — Iteration 3:

Trigger / Context	Action	Resulting Screen / Behavior
Home Screen	Press any key	→ Name Input Screen
Name Input Screen	Press Enter (non-empty name)	→ Main Menu Screen
Main Menu Screen	Select 'Start Game'	→ Game Screen Layout (Level 1)
Main Menu Screen	Select 'Instructions'	→ Instructions Screen
Main Menu Screen	Select 'High Scores'	→ High Score Screen
Main Menu Screen	Select 'Exit' or press ESC	→ Exit Program
Game Screen (Level 1)	All bricks broken	→ Level Complete Screen → [ENTER] → Level 2
Game Screen (Level 2)	All bricks broken	→ Level Complete Screen → [ENTER] → Level 3
Game Screen (Level 3)	All bricks broken	→ Win Screen → [ENTER/ESC] → Main Menu or Restart
Any Game Screen	Lives = 0	→ Game Over Screen → [ENTER] → Restart [ESC] → Main Menu

Trigger / Context	Action	Resulting Screen / Behavior
Any Game Screen	Press ESC	→ Main Menu Screen

4. Technical Architecture

4.1 Game Loop Architecture

The game loop is done within GameScreenLayout and is a tight hardware-polling loop that is repeated over and over. Loop is run after game objects are all created and will loop until ESC is pressed or lives are reached at zero. The same logic used for the loop is used for each of the three levels, only the brick state array and collision boundaries change for each level.

Step	Operation	Description
1	Read keyboard port	Reads hardware port 60h directly using IN AL, 60H. This bypasses BIOS INT 16h to allow holding keys for continuous paddle movement.
2	Check release bit	Bit 7 of the scan code is set on key-release events. If set, the frame skips to no-move handling, preventing ghost presses.
3	Handle input	ESC (01h) exits to menu. Scan code 4Bh / 1Eh moves left. Scan code 4Dh / 20h moves right. Scan code 39h (SPACE) launches the ball. F2 (3Ch) skips directly to Level 2 (debug shortcut).
4	Move paddle	ClearPaddle erases the old position; new X is calculated and clamped to screen bounds; DrawPaddle renders at the new position.
5	Move ball	If ball_launched = 1, MoveBall is called. If not launched, the ball tracks paddle_x + 32 and moves with it.
6	Update bonus	UpdateBonus erases the bonus at its old Y, moves it down 2px, checks paddle collection or out-of-bounds miss.

7	Tick bonus timer	TickBonusTimer decrements the active effect countdown; when it reaches zero, RestoreBonusEffect resets ball speed and paddle width.
8	Delay	A nested busy-wait loop burns clock cycles to control frame speed. Outer iteration count = ball_speed; smaller value = faster game.
9	Repeat	JMP back to gs_game_loop. Loop runs until ESC is pressed or lives reach zero.

4.2 Collision Detection

The MoveBall procedure takes care of all collision detection. The ball is drawn as a 6×6 pixel rectangle. Each frame, priority is checked, starting from the highest. Priority is checked from priority number 1 to priority number last each frame for collisions.

Wall Collisions

- Left wall: if ball_x < 2, negate ball_dx and clamp X to 2.
- Right wall: if ball_x + 5 > 317, negate ball_dx and clamp X to 311.
- Top wall (gameplay ceiling): if ball_y < 30, negate ball_dy and clamp Y to 30.
- Bottom boundary: if ball_y + 5 >= 199, a life is lost. There is intentionally no bounce — this is the lose condition.

Paddle Collision

Paddle collision is checked only when ball_dy > 0 (ball moving downward). All four conditions must be true for a paddle hit:

- ball_y + 5 >= paddle_y (ball bottom edge at or past paddle top)
- ball_y <= paddle_y (ball top edge not below paddle)
- ball_x >= paddle_x (ball left edge inside paddle left bound)
- ball_x < paddle_x + paddle_width (ball left edge inside paddle right bound)

When all four conditions are met, ball_dy is negated and ball_y is set to paddle_y - 6 to prevent the ball from sinking into the paddle.

Brick Collisions

Brick collision uses grid arithmetic rather than per-pixel checking. The active brick state array (brick_state, brick_state2, or brick_state3) is selected based on current_level.

- The brick grid starts at Y=48. Each row is 11px tall; each column is 20px wide, starting at X=10.
- Row index = (ball_y - 48) / 11 (integer division)
- Column index = (ball_x - 10) / 20 (integer division)
- Flat array index = row * 15 + col.
- If brick_state[index] = 1 (alive): mark as dead (0), erase screen pixels, add 5 to score, negate ball_dy, call SpawnBonus.
- If brick_state[index] = 0 (dead) or = 2 (permanent gap in Level 2): execution falls through to paddle collision checking.
- After each brick break, CheckAllBricksDead scans the full array; if all breakable bricks are dead, LevelCompleteScreen is called.

4.3 Multi-Level System

A multi-level system is controlled with the use of a dedicated set of transition procedures and current_level variable. The brick state array, layout drawing procedure and initialization sequence are different for each level.

Component	Level 1	Level 2	Level 3
State Array	brick_state (75 bytes)	brick_state2 (90 bytes)	brick_state3 (105 bytes)
Layout Proc	GameScreenLayout	DrawLevel2Bricks	DrawLevel3Bricks
Transition Proc	—	TransitionToLevel2	TransitionToLevel3
Grid Size	5 rows x 15 cols	6 rows x 15 cols	7 rows x 15 cols

Breakable Bricks	75	60 (tunnel gap)	~75 (fortress)
Ball Speed	6	4	2
Ball dx / dy	1 / -1	1 / -1	2 / -2

Transition Procedure

If CheckAllBricksDead returns true, then LevelCompleteScreen is called (which waits for player input), and the appropriate transition procedure is called. Each transition procedure:

- Sets current_level to the new value.
- Calls FillScreen to clear the display.
- Redraws the HUD with the updated level number.
- Calls the corresponding DrawLevelXBricks procedure to initialize the brick state array and render bricks.
- Resets bonus_active, bonus_timer, bonus_effect, brick_break_count, and paddle_width to defaults.
- Repositions and redraws the paddle and ball, then returns to the game loop.

4.4 Bonus / Power-Up System

Every third brick break, the Bonus System gives a falling colored piece. Player can collect the bonus by running the paddle underneath. There can be only one bonus active (falling) at any one time, and there can be only one effect active at any one time.

Spawn Logic (SpawnBonus)

- brick_break_count is incremented each time a brick is destroyed.
- When brick_break_count mod 3 equals 0, a bonus is spawned (bonus_active = 1).
- bonus_type is determined by (brick_break_count / 5) mod 5, cycling through types 1–5.
- Initial position: bonus_x = 155 (mid-screen), bonus_y = 120.

Falling Animation (UpdateBonus)

- Called every frame from the main game loop.
- Erases the bonus square at its current position with a black FillRect.
- Increments bonus_y by 2 (falling at 2 pixels per frame).
- If bonus_y + 8 >= paddle_y: checks for horizontal overlap with the paddle.
- If bonus_y >= 200: cancels the bonus (bonus_active = 0) — the bonus was missed.

Bonus Types and Effects

Type	Color	Effect	Duration
1: Slow Ball	Cyan (03h)	ball_speed set to 10 (slower)	500 frames
2 : Fast Ball	Red (04h)	ball_speed set to 2 (faster)	500 frames
3 : Extra Life	Green (02h)	+1 to lives_count (max 5)	Permanent
4 : Wide Paddle	Yellow (0Eh)	paddle_width set to 100	600 frames
5 : Narrow Paddle	Magenta (0Dh)	paddle_width set to 35	600 frames

Effect Expiry (TickBonusTimer / RestoreBonusEffect)

- bonus_timer is decremented each frame when bonus_effect != 0.
- When bonus_timer reaches 0, RestoreBonusEffect is called automatically.
- RestoreBonusEffect: resets ball_speed to 6, redraws paddle at paddle_width = 70, clears bonus_effect and bonus_timer.
- If a new bonus is collected while an effect is active, RestoreBonusEffect is called first, then the new effect is applied.

4.5 Lives System

The lives system manages the three-life limit and ball reset behaviour. `lives_count` stores the current lives (initial value: 3).

Life Decrement

- When `ball_y + 5 >= 199` (ball exits below screen), `MoveBall` decrements `lives_count` by 1.
- `UpdateLivesDisplay` is called immediately to redraw the HUD.
- If `lives_count` reaches 0, `GameOverScreen` is called. Registers are restored before the call to ensure a clean stack.

Ball Reset (Lives > 0)

- `ball_launched` is set to 0.
- `ball_x` is reset to `paddle_x + 32` (centered on paddle).
- `ball_y` is reset to 170. `ball_dx` and `ball_dy` are reset to 1 and -1 respectively.
- The ball is redrawn in white (0Fh) at the reset position.
- The player must press SPACE again to re-launch, giving time to position the paddle.

4.6 Sound Effects System

The 8253 Programmable Interval Timer (PIT) is used to make sound produced through the PC speaker. `SimpleBeep`: Programs timer channel 2 and enables speaker on port 61h. All sound calls are non-blocking, meaning that they are executed in a short loop, with the `sound_freq` and `sound_duration` variables.

Procedure	Trigger	Description
<code>PlayPaddleHit</code>	Ball hits paddle	Two-tone descending click (800Hz → 600Hz)
<code>PlayGlassBreak</code>	Brick destroyed	Four-tone descending shatter (1200 → 1000 → 800 → 600Hz)

PlayBonusSound	Bonus collected	Four-tone ascending bubble (600 → 700 → 800 → 900Hz)
PlayWinCheer	Level complete	Ascending triumphant melody (330 → 392 → 440 → 523Hz)
PlayLossSound	Game over / restart	Descending sad melody (440 → 392 → 349 → 330Hz)
PlayMenuSound	Menu navigation	Single brief beep (1000Hz)
PlayMenuConfirm	Menu selection	Two-tone confirm (800 → 1000Hz)
PlayGameStart	Game begins	Rising scale (400 → 500 → 600 → 800Hz)
PlayGameOverSound	Game Over screen	Dramatic low drone (200 → 300 → 250 → 200Hz)
PlayFanfare	Level complete fanfare	High-register fanfare (659 → 784 → 880 → 987Hz)

5. Iteration 3 Scope & Marking

The following tables summarise the requirements for Iteration 3 and the implementation status of each component.

5.1 Multi-Level System (10 Marks)

Component	Marks	Status
Level 2 implemented (Tunnel Layout, 6 rows x 15 cols)	2	Implemented
Level 3 implemented (Fortress Layout, 7 rows x 15 cols)	2	Implemented
Speed increase per level (ball_speed: 6 → 4 → 2; ball_dx/dy: 1 → 2)	2	Implemented

Different brick layouts per level (uniform → tunnel gap → fortress)	2	Implemented
Smooth transition between levels (LevelCompleteScreen + TransitionToLevelX)	2	Implemented

5.2 Power-Up System (10 Marks)

Component	Marks	Status
Random spawn logic (every 3rd brick break via brick_break_count mod 3)	2	Implemented
Falling animation (UpdateBonus moves bonus down 2px per frame)	2	Implemented
Paddle collision detection (overlap check in UpdateBonus)	2	Implemented
At least 3 working bonus types (5 types: Slow Ball, Fast Ball, Extra Life, Wide Paddle, Narrow Paddle)	4	Implemented

5.3 Complete Game Flow (5 Marks)

Component	Marks	Status
Level progression works (L1 → L2 → L3 via CheckAllBricksDead)	2	Implemented

Game Over screen (triggered when lives_count = 0 in MoveBall)	1	Implemented
Win / Completion screen (LevelCompleteScreen with is_final_win = 1)	1	Implemented
Return to menu / replay option (ESC → menu, ENTER → restart on all end screens)	1	Implemented

5.4 Game Polish & Stability (5 Marks)

Component	Marks	Status
No crashes / freezes (register save/restore on all procedures, clean stack discipline)	2	Implemented
Smooth rendering (erase-then-redraw pattern; no full-screen flicker)	1	Implemented
Balanced gameplay (three distinct speed tiers; bonus effects with timed expiry)	1	Implemented
Clean transitions (LevelCompleteScreen pause + full HUD redraw on transition)	1	Implemented

5.5 Bonus Marks (+10)

Feature	Marks Available	Status
File handling (save/load high scores)	+3	Not implemented
Pause system (key press to pause/resume)	+2	Not implemented

Sound effects (PC speaker)	+2	Implemented
Mouse support	+2	Not implemented
Visual effects (animations/trails)	+1	Implemented

27. Data Structures

The following table describes all major variables declared in the .DATA segment of the program, their type, purpose, and size.

27.1 Paddle & Ball Variables

Variable	Type	Size	Purpose
paddle_x	DW	2 bytes	Current X position of the paddle's left edge.
paddle_y	DW	2 bytes	Fixed Y position of the paddle (always 185).
paddle_width	DW	2 bytes	Current width of the paddle in pixels (default 70; changed by bonuses).
paddle_old_x	DW	2 bytes	Previous X position, used to erase paddle before redraw.
paddle_speed	DW	2 bytes	Number of pixels the paddle moves per key press (10).
ball_x	DW	2 bytes	Current X position of the ball's top-left corner.
ball_y	DW	2 bytes	Current Y position of the ball's top-left corner.
ball_dx	DW	2 bytes	Horizontal velocity of the ball (+1 or -1; +2/-2 in Level 3).

ball_dy	DW	2 bytes	Vertical velocity of the ball (+1 or -1; +2/-2 in Level 3).
ball_speed	DW	2 bytes	Delay loop count controlling frame rate (6=normal, 4=L2, 2=L3).
ball_launched	DB	1 byte	Flag: 0 = ball sitting on paddle; 1 = ball in flight.

27.2 Brick State Arrays

Variable	Type	Size	Purpose
brick_state	DB	75 bytes	Level 1 brick states (5 rows x 15 cols). 1 = alive, 0 = dead.
brick_state2	DB	90 bytes	Level 2 brick states (6 rows x 15 cols). 1 = alive, 0 = dead, 2 = permanent gap.
brick_state3	DB	105 bytes	Level 3 brick states (7 rows x 15 cols). 1 = alive, 0 = dead.

27.3 Game State Variables

Variable	Type	Size	Purpose
lives_count	DW	2 bytes	Number of lives remaining (starts at 3, max 5 with bonus).
score	DW	2 bytes	Current player score (incremented by 5 per brick broken).
current_level	DW	2 bytes	Active level number (1, 2, or 3).
game_running	DB	1 byte	General game state flag (reserved for future use).
is_final_win	DB	1 byte	0 = level complete screen; 1 = final win (all 3 levels cleared).
temp_str	DB	10 bytes	Scratch buffer used for converting numbers to ASCII for display.

27.4 Bonus / Power-Up Variables

Variable	Type	Size	Purpose
bonus_active	DB	1 byte	1 = a bonus item is currently falling; 0 = none active.
bonus_x	DW	2 bytes	Current X position of the falling bonus square.

bonus_y	DW	2 bytes	Current Y position of the falling bonus square.
bonus_type	DB	1 byte	Type of the falling bonus: 1=Slow, 2=Fast, 3=Life, 4=Wide, 5=Narrow.
bonus_timer	DW	2 bytes	Frame countdown for timed effects (500 or 600 frames).
bonus_effect	DB	1 byte	Currently applied effect (0=none; mirrors bonus_type of last collected).
brick_break_count	DB	1 byte	Running count of bricks broken; bonus spawns every 3rd break.

27.5 String & Display Buffers

Variable	Type	Size	Purpose
name_buf	DB	21 bytes	Raw input buffer for the player's name (null-terminated, max 20 chars).
name_stored	DB	21 bytes	Stored copy of the player name after confirmation.
gs_name	DB	21 bytes	Copy of player name used in the in-game HUD display.
gs_scoreDisplayStr	DB	11 bytes	'SCORE: 0000' -- last 4 bytes updated by UpdateScoreString.
gs_livesDisplayStr	DB	6 bytes	Static label 'LIVES:' drawn in HUD.
gs_levelDisplayStr	DB	9 bytes	Static label 'LEVEL: 01' drawn in HUD (Level 1 only).
name_len	DW	2 bytes	Current length of text in name_buf.

27.6 Font Table

Variable	Type	Size	Purpose
font_table	DB	8 bytes x 48 glyphs = 384 bytes	Bitmap font. Each glyph is 8 rows x 8 columns (1 bit per pixel). Indexed by CharToIndex. Covers A-Z, 0-9, and special symbols including heart, diamond, and circle icons used for HUD lives display.

27.7 Sound Variables

Variable	Type	Size	Purpose
sound_active	DB	1 byte	Reserved flag for sound state tracking (not actively used in current logic).
sound_duration	DW	2 bytes	Outer loop count passed to SimpleBeep; controls tone duration.
sound_freq	DW	2 bytes	Frequency divisor written to PIT port 42h; lower = higher pitch.
PADDLE_HIT_FREQ	DW	2 bytes	Constant: 800 Hz reference for paddle hit sound.
GLASS_BREAK_FREQ	DW	2 bytes	Constant: 1200 Hz reference for brick break sound.
BONUS_FREQ	DW	2 bytes	Constant: 1000 Hz reference for bonus collect sound.

28. Procedure Descriptions

The following table lists every procedure in the program with a 2-3 sentence description of its role, inputs, and outputs.

28.1 Screen Procedures

Procedure	Description
HomeScreen	Fills the screen black, draws the animated brick background, renders the centered BRICK BREAKERS title box with a double-rectangle border, and displays team credits at the bottom. Waits for any key press (INT 16h) before returning.
NameInputScreen	Draws the name entry UI -- title, separator lines, input box, and instruction footer. Loops reading characters from INT 16h, updating name_buf and calling UpdateInputDisplay on each keystroke, until Enter is pressed with a non-empty buffer.
UpdateInputDisplay	Clears the input box area with a black FillRect then redraws the current contents of name_buf in white text. Called after every keystroke in NameInputScreen.
MainMenuScreen	Initialises menu state variables, calls DrawMainMenuStatic, then enters a polling loop that handles UP/DOWN navigation, Enter to select, and ESC to exit. Calls AnimateMenuBall on every iteration to bounce the decorative ball.
DrawMainMenuStatic	Fills the screen, draws the neon double-border frame via DrawNeonFrame, renders the MAIN MENU title and underline, calls DrawAllMenuOpts, and draws the paddle rectangle in the lower-right corner.
DrawAllMenuOpts	Renders all four menu option strings; the currently selected option is drawn in magenta (0Dh), all others in white (0Fh).
RedrawMenuHighlight	Erases the previous selection highlight by redrawing the old option in white, then draws the new selection in magenta with a '>' arrow. More efficient than a full menu redraw.
DrawMenuTextAt	Helper used by RedrawMenuHighlight. Draws the '>' arrow before the text if the color is magenta, otherwise clears the arrow region with a black FillRect, then calls DrawString.
AnimateMenuBall	Erases the menu ball at its current position, increments ball_my by ball_mdy, bounces off top (Y=12) and bottom (Y=166) boundaries, redraws the ball in magenta, and calls MediumDelay.
InstructionsScreen	Draws page 1 (Controls, Objective, Lives) or page 2 (Bonuses table) depending on state. Enter toggles between pages; ESC returns to the caller (MainMenuScreen). Both pages share the same neon frame and title.
HighScoreScreen	Draws the HIGH SCORES screen with a static five-entry leaderboard. Each entry is color-coded by rank. Waits for any key press then returns.

28.2 Game Screen Procedures

Procedure	Description
GameScreenLayout	Initialises all game state variables (score, lives, level, bonuses, bricks), draws the HUD, renders Level 1 bricks, positions the paddle and ball, then enters the main game loop (gs_game_loop). The loop polls port 60h for keyboard input, calls MoveBall and UpdateBonus each frame, and applies a speed-controlled delay.
TransitionToLevel2	Sets current_level = 2, redraws the full screen and HUD with 'LEVEL: 02', calls DrawLevel2Bricks to initialise brick_state2 and render the tunnel layout, resets all bonus and ball state, and returns to the game loop.
TransitionToLevel3	Sets current_level = 3, redraws the full screen and HUD with 'LEVEL: 03', calls DrawLevel3Bricks to initialise brick_state3 and render the fortress layout, increases ball speed (ball_speed=2, ball_dx/dy=2), and returns to the game loop.
DrawLevel2Bricks	Initialises brick_state2 with value 1 for columns 0-5 and 9-14, and value 2 for gap columns 6-8. Then renders a 6-row x 15-column grid, painting gap columns black and alternating row colors between cyan (03h) and magenta (0Dh).
DrawLevel3Bricks	Initialises brick_state3 to 0, then sets alive cells for the fortress pattern (outer walls + inner block). Renders each alive cell with a color chosen by position: dark blue or magenta for outer walls, magenta for top/bottom rows, cyan for inner block.
DrawPaddle	Saves all registers, then calls FillRect with color 0Fh (white) at paddle_x, paddle_y, paddle_width x 8 to render the paddle.
ClearPaddle	Saves all registers, then calls FillRect with color 00h (black) at the current paddle position to erase it before movement.

28.3 Game Logic Procedures

Procedure	Description
MoveBall	The core physics procedure. Erases the ball, calculates new X/Y from ball_dx/ball_dy, checks wall collisions (left, right, top), then checks brick collisions using grid arithmetic against the active brick state array, and finally checks paddle collision. On a brick hit: marks brick dead, erases it, increments score, negates ball_dy, calls SpawnBonus and CheckAllBricksDead. On a bottom exit: decrements lives, calls GameOverScreen if zero, otherwise resets ball to paddle.
CheckAllBricksDead	Scans the appropriate brick state array for the current level. If all breakable cells are 0, sets is_final_win appropriately, calls LevelCompleteScreen, then calls TransitionToLevel2 or TransitionToLevel3 (or returns for the win state). Uses early exit (je cabd_not_done) to abort the scan as soon as a live brick is found.

GameOverScreen	Draws the GAME OVER screen with stats (score, level, bricks left, lives). Loops on INT 16h: ENTER resets all game state and calls GameScreenLayout; ESC returns to MainMenuScreen.
LevelCompleteScreen	Draws either the LEVEL COMPLETE or YOU WIN screen depending on is_final_win. Displays score, level reached, bricks left, and lives remaining. ENTER proceeds (next level or full restart); ESC returns to menu.

28.4 Bonus / Power-Up Procedures

Procedure	Description
SpawnBonus	Increments brick_break_count. If the count is divisible by 3 and no bonus is currently falling, calculates bonus_type (1-5 cycling), sets bonus position to (155, 120), sets bonus_active = 1, and calls DrawBonus.
DrawBonus	Selects a fill color based on bonus_type (cyan/red/green/yellow/magenta) and calls FillRect to render the 8x8 bonus square at bonus_x, bonus_y.
UpdateBonus	Called every frame. Erases the bonus at its current position, advances bonus_y by 2 pixels, checks if the bonus bottom edge has reached paddle_y and overlaps the paddle horizontally (collection), or if bonus_y >= 200 (miss). On collection: sets bonus_active = 0, calls ApplyBonus and PlayBonusSound. On miss: sets bonus_active = 0. Otherwise redraws the bonus at the new position.
ApplyBonus	Calls RestoreBonusEffect to cancel any existing effect, then applies the new effect based on bonus_type: sets ball_speed (types 1 and 2), increments lives (type 3), or changes paddle_width and redraws the paddle (types 4 and 5). Sets bonus_effect and bonus_timer accordingly.
TickBonusTimer	Called every frame. If bonus_effect != 0 and bonus_timer > 0, decrements bonus_timer. When bonus_timer reaches 0, calls RestoreBonusEffect.
RestoreBonusEffect	Resets ball_speed to 6, redraws the paddle at width 70 (clearing the old size first), and clears bonus_effect and bonus_timer to 0.

28.5 Drawing Utility Procedures

Procedure	Description
FillScreen	Writes color AL to all 64,000 bytes of VGA segment A000h using REP STOSB. Used to clear the screen at the start of every new screen.
DrawHLine	Draws a horizontal line of CX pixels at position (BX, DX) using color AL. Calculates the VGA memory offset as DX*320 + BX and uses REP STOSB.
FillRect	Fills a rectangle by calling DrawHLine once per row for DI rows, starting at (BX, DX) with width SI and color AL.
DrawRect	Draws only the border of a rectangle (4 sides) using DrawHLine for top/bottom and single-pixel horizontal calls for the left and right edges.

DrawVLine	Draws a vertical line of CX pixels at position (BX, DX) by writing AL to VGA memory with a stride of 320 bytes per step.
DrawNeonFrame	Draws two concentric border rectangles -- outer purple (05h) at (4,4) with 312x190, and inner blue (01h) at (7,7) with 306x184 -- to create the neon glow frame used on menu and informational screens.
DrawBricks	Draws the decorative background brick grid used on the Home and Name Input screens (4 rows x 16 columns, color-cycling).
CharToIndex	Converts an ASCII character (letter, digit, or symbol) to a font table index (0-47). Returns the index in AX. Handles A-Z (case-insensitive), 0-9, and all special characters defined in the font table.
DrawChar	Renders a single 8x8 bitmap character at position (BX, DX) using color CH. Calls CharToIndex to find the glyph, then iterates over 8 rows x 8 columns, writing a pixel for each set bit.
DrawString	Renders a null-terminated string by repeatedly calling DrawChar and advancing BX by 8 per character. Uses SI as the source pointer (LODSB).

28.6 HUD Update Procedures

Procedure	Description
UpdateScoreString	Converts the 16-bit score variable to a 4-digit ASCII string and writes it into bytes 7-10 of gs_scoreDisplayStr using division by 1000, 100, 10.
RefreshScoreHUD	Clears the score area in the top bar with a purple FillRect, then redraws gs_scoreDisplayStr in white using DrawString.
UpdateLivesDisplay	Clears the lives area in the top bar, then draws up to 5 red heart icons (ASCII value 3, color 0Ch) corresponding to the current lives_count value.

28.7 Sound Procedures

Procedure	Description
SimpleBeep	Programs the 8253 PIT timer 2 (ports 43h and 42h) with the divisor in sound_freq, enables the PC speaker via port 61h, runs a delay loop of sound_duration outer iterations, then disables the speaker. This is the low-level tone engine called by all higher-level sound procedures.
PlayPaddleHit	Plays a two-tone descending click (800Hz then 600Hz) to indicate ball-paddle contact.
PlayGlassBreak	Plays a four-tone descending sequence (1200 to 1000 to 800 to 600Hz) to simulate a brick shattering.
PlayBonusSound	Plays a four-tone ascending bubble sequence (600 to 900Hz) to indicate a bonus has been collected.

PlayWinCheer	Plays an ascending triumphant four-note melody (330 to 523Hz) for level completion.
PlayLossSound	Plays a descending sad four-note melody (440 to 330Hz) for game over and screen transitions.
PlayMenuSound	Plays a single brief 1000Hz beep for menu cursor movement.
PlayMenuConfirm	Plays a two-tone ascending confirm sound (800 to 1000Hz) for menu selection.
PlayGameStart	Plays a rising four-note scale (400 to 800Hz) when a game session begins.
PlayGameOverSound	Plays a dramatic low drone pattern (200 to 300 to 250 to 200Hz) looped while waiting on the Game Over screen.
PlayFanfare	Plays a high-register four-note fanfare (659 to 987Hz) for level completion celebration.

28.8 Delay Procedures

Procedure	Description
MediumDelay	Busy-wait loop of 12,000 iterations. Used in AnimateMenuBall to throttle the menu animation speed.
SmallDelay	Busy-wait loop of 8,000 iterations. Available as a shorter delay utility for use where MediumDelay is too slow.

29. Known Issues

The following limitations and bugs are known in the current submission. They are documented honestly in accordance with the documentation requirements.

#	Issue	Affected Area	Details
1	Sound causes minor frame rate stutter	Sound Effects	SimpleBeep is a blocking, synchronous call. During multi-tone sounds (PlayGlassBreak, PlayFanfare), the game loop is briefly paused, which can cause a perceptible stutter on slower DOSBox configurations.

30. Task Division

The following table shows which team member was primarily responsible for each component of the project across all three iterations.

ERMISH:

- Iteration 1: home page, name page, font bitmap
- Iteration 2: Paddle mechanics and input handling,
- Iteration 3: level 3, win screen, sound affects

SAWAIRA:

- Iteration 1: integration, instructions screen, main menu screen
- Iteration 2: game over screen, integration and documentation
- Iteration 3: polish and error handling, screen flow diagram

HANZALA:

- Iteration 1: high score screen, level 1 brick layout
- Iteration 2: ball mechanics
- Iteration 3: video documentation

AHMED:

- Iteration 1: video and documentation
- Iteration 2: video, brick system and HUD, life management
- Iteration 3: level 2 and bonus system